LAB 5 | Batch vs stochastic vs mini-batch

ANN, MLP AND CNN COMPARISON

Keep the SGD optimizer fixed and change only the fit() batch_size to demonstrate how the same architecture trains with full-batch, single-example, or mini-batch updates.

Learning objectives

- Explain the distinction between optimizer choice and batch-regime choice.
- Calculate total weight updates in three epochs for the three regimes.
- Compare measured loss, validation accuracy and runtime for nine runs.

Reproducible experiment settings

- Fashion-MNIST | training 600 | validation 200 | test 500.
- ANN (one hidden layer), MLP (two hidden layers), CNN (two convolution blocks).
- Keep SGD(learning_rate=0.02), EPOCHS=3 and initial weights fixed within each model.
- Change BATCH_SIZE only: full batch=600, stochastic=1, mini-batch=32.

Run instructions

1. In Google Colab, create a notebook, paste the complete Python solution and run it.
2. For local Python, install TensorFlow, NumPy and Matplotlib: pip install tensorflow numpy matplotlib
3. The public dataset downloads on the first execution. Actual performance numbers are created only when the code is run.
4. The Python (.py) source file is provided separately for direct execution and easier copying.

Complete worked solution | line-commented Python code

Instructional comments accompany each executable statement. The standalone .py file contains exactly this code; output numbers and charts are generated by running it.

```python
# LAB 3: Compare full-batch, stochastic, and mini-batch gradient descent.
# Run in Google Colab or locally after: pip install tensorflow numpy matplotlib
# Models: one-hidden-layer ANN, two-hidden-layer MLP, and compact CNN.
# Key control: use the SAME tf.keras.optimizers.SGD(learning_rate=0.02) everywhere.
# The gradient descent technique is selected by model.fit(..., batch_size=...).

import os  # Control the quantity of TensorFlow startup messages.
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"  # Reduce routine TensorFlow logging.
import csv  # Create a table students can open in Excel.
import math  # Calculate the number of weight updates per epoch.
import time  # Time each training setup on the current machine.
from pathlib import Path  # Make portable filenames and result folders.
import numpy as np  # Prepare reproducible subsets of image examples.
import matplotlib.pyplot as plt  # Visualize measured validation accuracy.
import tensorflow as tf  # Build and optimize all three neural networks.

SEED = 42  # Specify a reproducible dataset selection and random seed.
TRAIN_N = 600  # Keep single-example SGD feasible during a short classroom lab.
VAL_N = 200  # Reserve 200 images for validation after every epoch.
TEST_N = 500  # Use held-out test images after training each run.
EPOCHS = 3  # Use the same number of complete data passes in every run.
LEARNING_RATE = 0.02  # Hold the SGD learning rate constant across techniques.
OUT = Path("lab3_gradient_descent_results")  # Select a folder for CSV data and figures.
OUT.mkdir(parents=True, exist_ok=True)  # Create the output folder if it is missing.
tf.keras.utils.set_random_seed(SEED)  # Seed supported Python and machine-learning RNGs.

(x_all, y_all), (x_test_all, y_test_all) = tf.keras.datasets.fashion_mnist.load_data()  # Load public clothes images.
rng = np.random.default_rng(SEED)  # Set an independent reproducible NumPy random source.
chosen_ids = rng.permutation(len(x_all))[:TRAIN_N + VAL_N]  # Randomly select 800 development examples.
test_ids = rng.permutation(len(x_test_all))[:TEST_N]  # Randomly select separate test examples.
x_train = x_all[chosen_ids[:TRAIN_N]].astype("float32") / 255.0  # Scale training images.
y_train = y_all[chosen_ids[:TRAIN_N]]  # Select training target classes.
x_val = x_all[chosen_ids[TRAIN_N:]].astype("float32") / 255.0  # Scale validation images.
y_val = y_all[chosen_ids[TRAIN_N:]]  # Select validation target classes.
x_test = x_test_all[test_ids].astype("float32") / 255.0  # Scale final held-out images.
y_test = y_test_all[test_ids]  # Select their correct class labels.

def build_ann():  # Make the shallow fully connected neural network.
    return tf.keras.Sequential([  # Construct layers in sequential order.
        tf.keras.layers.Input(shape=(28, 28)),  # Accept a 28-by-28 image.
        tf.keras.layers.Flatten(),  # Transform image pixels into a vector.
        tf.keras.layers.Dense(64, activation="relu"),  # Add one hidden Dense layer.
        tf.keras.layers.Dense(10, activation="softmax"),  # Predict 10 output classes.
    ])  # Complete the shallow ANN.

def build_mlp():  # Make a deeper multilayer perceptron architecture.
    return tf.keras.Sequential([  # Construct a fully connected model.
        tf.keras.layers.Input(shape=(28, 28)),  # Accept a grayscale image.
        tf.keras.layers.Flatten(),  # Flatten input into 784 numeric features.
        tf.keras.layers.Dense(128, activation="relu"),  # Add the first hidden layer.
        tf.keras.layers.Dense(64, activation="relu"),  # Add the second hidden layer.
        tf.keras.layers.Dense(10, activation="softmax"),  # Predict clothing classes.
```

```
    ])  # Complete the MLP.

def build_cnn():  # Build a compact convolutional classifier.
    return tf.keras.Sequential([  # Use the same framework as the other models.
        tf.keras.layers.Input(shape=(28, 28, 1)),  # Expect an explicit grayscale channel.
        tf.keras.layers.Conv2D(16, 3, activation="relu"),  # Learn basic visual features.
        tf.keras.layers.MaxPooling2D(),  # Reduce feature-map sizes.
        tf.keras.layers.Conv2D(32, 3, activation="relu"),  # Learn higher-level patterns.
        tf.keras.layers.MaxPooling2D(),  # Downsample a second time.
        tf.keras.layers.Flatten(),  # Transform feature maps into a vector.
        tf.keras.layers.Dense(32, activation="relu"),  # Combine visual features.
        tf.keras.layers.Dense(10, activation="softmax"),  # Output class probabilities.
    ])  # Complete the CNN network.

MODELS = [("ANN", build_ann), ("MLP", build_mlp), ("CNN", build_cnn)]  # List three model designs.
TECHNIQUES = [("Full batch", TRAIN_N), ("Stochastic", 1), ("Mini-batch", 32)]  # Change batch_size only.
results = []  # Collect the results of three models times three regimes.
for model_name, builder in MODELS:  # Evaluate each network architecture in turn.
    if model_name == "CNN":  # CNN convolution requires four-dimensional image tensors.
        x_tr = x_train[..., np.newaxis]  # Add one channel to training images.
        x_va = x_val[..., np.newaxis]  # Add one channel to validation images.
        x_te = x_test[..., np.newaxis]  # Add one channel to testing images.
    else:  # Fully connected networks accept the original 28-by-28 images.
        x_tr, x_va, x_te = x_train, x_val, x_test  # Reuse the same images without channel expansion.
    tf.keras.utils.set_random_seed(SEED)  # Reproduce this network's original initialization.
    reference = builder()  # Construct a new, as-yet-untrained reference model.
    initial_weights = reference.get_weights()  # Save the identical starting parameters.
    del reference  # Free the no-longer-needed reference network.
    histories = {}  # Store each technique's true validation accuracy curve.
    for technique, batch_size in TECHNIQUES:  # Change only the batch size within this model.
        tf.keras.utils.set_random_seed(SEED)  # Reset randomness between independent runs.
        model = builder()  # Rebuild the very same architecture.
        model.set_weights(initial_weights)  # Reset exactly the same starting weights.
        optimizer = tf.keras.optimizers.SGD(learning_rate=LEARNING_RATE)  # Fix the optimizer AND learning rate.
        model.compile(optimizer=optimizer, loss="sparse_categorical_crossentropy", metrics=["accuracy"])  # Prepare supervised learning.
        steps_per_epoch = math.ceil(TRAIN_N / batch_size)  # Count expected updates per epoch.
        start = time.perf_counter()  # Begin timing the actual training run.
        history = model.fit(x_tr, y_train, validation_data=(x_va, y_val), epochs=EPOCHS, batch_size=batch_size, shuffle=True, verbose=0)  # Run this gradient regime.
        elapsed = time.perf_counter() - start  # Record elapsed wall-clock training time.
        test_loss, test_accuracy = model.evaluate(x_te, y_test, verbose=0)  # Evaluate on held-out examples.
        best_val = max(history.history["val_accuracy"])  # Check the strongest validation epoch.
        histories[technique] = history.history["val_accuracy"]  # Save the actual accuracy trajectory.
        results.append({"model": model_name, "method": technique, "batch_size": batch_size, "updates_per_epoch": steps_per_epoch, "total_updates": EPOCHS * steps_per_epoch,
"best_val_accuracy": round(best_val, 4), "test_accuracy": round(test_accuracy, 4), "test_loss": round(test_loss, 4), "train_seconds": round(elapsed, 2)})  # Save metrics with update counts.
        print(f"{model_name:3} | {technique:10} | batch={batch_size:3} | steps/epoch={steps_per_epoch:3} | val={best_val:.3f} | test={test_accuracy:.3f}")  # Show observations.
    plt.figure(figsize=(8, 4.5))  # Create the current network's accuracy plot.
    for method, curve in histories.items():  # Compare the three real training histories.
        plt.plot(range(1, EPOCHS + 1), curve, marker="o", label=method)  # Plot observed accuracies.
    plt.title(f"{model_name}: validation accuracy by batch regime")  # Identify current architecture.
    plt.xlabel("Epoch")  # Put complete dataset passes on the horizontal axis.
    plt.ylabel("Validation accuracy")  # Show validation performance on the vertical axis.
    plt.xticks(range(1, EPOCHS + 1))  # Display integer epoch tick marks.
    plt.grid(alpha=0.25)  # Make slopes easier to compare.
    plt.legend()  # Label full-batch, stochastic, and mini-batch curves.
    plt.tight_layout()  # Avoid clipping axis annotations.
    plt.savefig(OUT / f"{model_name.lower()}_batch_methods.png", dpi=160)  # Save current model chart.
    plt.close()  # Release this figure's memory.

with (OUT / "batch_method_results.csv").open("w", newline="", encoding="utf-8") as file:  # Open results CSV file.
```

```
    writer = csv.DictWriter(file, fieldnames=list(results[0].keys()))  # Define the CSV columns.
    writer.writeheader()  # Save informative table headings.
    writer.writerows(results)  # Save all nine measured runs.
print("CAUTION: Equal epochs do NOT mean equal updates when batch size differs.")  # Reinforce a key comparison limitation.
print(f"Results and plots saved to: {OUT.resolve()}")  # Show the location of output files.
```

Expected generated files

- lab3_gradient_descent_results/batch_method_results.csv
- lab3_gradient_descent_results/ann_batch_methods.png
- lab3_gradient_descent_results/mlp_batch_methods.png
- lab3_gradient_descent_results/cnn_batch_methods.png

How to interpret your results (sample answer framework)

Why do updates differ?

Which regime is fastest?

How to interpret accuracy

What is a stronger follow-up?

Practical cautions and further reading